**A Project Report**

**On**

# AIRFARE PRICES PREDICTION USING MACHINE LEARNING

**Submitted in partial fulfillment of the requirements for the award of the degree of**

## BACHELOR OF TECHNOLOGY
## IN
## DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

**By**


**S. ROHITH KUMAR**
**(228H1A5448)**

**M. VENKATA RAMAIAH**
**(228H1A5430)**

**K.N.V. SURYA**
**(228H1A5425)**

**A. NARENDRA**
**(228H1A5402)**


*Under the Esteemed Guidance of*

**Mr. S.VENKATESWARA RAO, M.Tech**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**




**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

## MVR College of Engineering & Technology

## (AUTONOMOUS)

**(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008, Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)**

**Paritala, NTR Dist., PIN 521180 A.P. India.**

**2025-2026**

**A Project Report**

**On**

# AIRFARE PRICES PREDICTION USING MACHINE LEARNING

**Submitted in partial fulfillment of the requirements for the award of the degree of**

## BACHELOR OF TECHNOLOGY
## IN
## DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

**By**


**S. ROHITH KUMAR**
**(228H1A5448)**

**M. VENKATA RAMAIAH**
**(228H1A5430)**

**K.N.V. SURYA**
**(228H1A5425)**

**A. NARENDRA**
**(228H1A5402)**


*Under the Esteemed Guidance of*

**Mr. S.VENKATESWARA RAO, M.Tech**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**




**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

## MVR College of Engineering & Technology

## (AUTONOMOUS)

**(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008,**
**Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)**
**Paritala, NTR Dist., PIN 521180 A.P. India.**
**2025-2026**

# MVR College of Engineering & Technology

# (AUTONOMOUS)

**(Approved by AICTE, Permanently Affiliated to JNTUK, Certified By ISO: 9001-2008, Accredited by NBA (CIVIL, CSE) & Accredited by NAAC with 'A' Grade)**

**Paritala, NTR Dist., PIN 521180 A.P. India.**

## DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

## CERTIFICATE




This is to certify that the Project Report entitled **"AIR FARE PRICES PREDICTION USING MACHINE LEARNING"** is being submitted by **"S. ROHITH KUMAR (228H1A5448), K.N.V. SURYA (228H1A5425), A. NARENDRA (228H1A5402), M. VENKATA RAMAIAH (228H1A5430)"** in partial fulfillment for the award of the Degree of Bachelor of Technology in Department of Computer Science & Engineering of MVR College of Engineering & Technology, for the record of bonafide work carried out by them.

**Mr. S.VENKATESWARA RAO**
**Assistant Professor**
**Project Supervisor**

**Mr. CH.SREENU BABU**
**Associate Professor**
**Head of the Department**

**EXAMINER**

# ACKNOWLEDGEMENT

I express my deep sense of gratitude and indebtedness to the **Management & Dr.U.Yedukondalu, Principal** of our college, who gave very good support to me during this work and others who assisted me in this project work.

I also thank **Mr. CH. SREENU BABU, Head of the Department of Computer Science Engineering**, who gave very good support to me during this work.

I also thank my **Project Supervisor**, **Mr. S. VENKATESWARA RAO, Asst. Professor** of **Computer Science & Engineering Department,** MVR College of Engineering & Technology, for her valuable guidance during the course of this project work. I am much indebted to her for suggesting a challenging and interactive project and her valuable advice at every stage of this work. I am very much thankful to her for her coordination in this regard.

Similarly, I am grateful to my friends I have worked with and consider myself extremely privileged and fortunate to work with such honest, hard working and loyal friends.

I wish my warm and grateful thanks to the staff of our department of **MVR College of Engineering & Technology** for the assistance for their continuous encouragement in completing my work successfully.

Finally I thank one and all who directly or indirectly helped me in completing project successfully.

**S. ROHITH KUMAR**
**(228H1A5448)**

**M. VENKATA RAMAIAH**
**(228H1A5430)**

**K.N.V. SURYA**
**(228H1A5425)**

**A. NARENDRA**
**(228H1A5402)**

# DECLARATION

I hereby declare that this project entitled **"AIRFARE PRICES PREDICTION USING MACHINE LEARNING"** has been undertaken by me and this work has been submitted to **MVR college of Engineering & Technology affiliated to JNTUK, Kakinada**, in partial fulfillment of the requirements for award of the degree of **Bachelor of Technology** in **Department of Computer Science & Engineering.**

I further declare that this project work has not been submitted in full or part for the award of any degree of this in any other educational institutions.


**S. ROHITH KUMAR**
**(228H1A5448)**

**M. VENKATA RAMAIAH**
**(228H1A5430)**

**K.N.V. SURYA**
**(228H1A5425)**

**A. NARENDRA**
**(228H1A5402)**

## LIST OF CONTENTS


| S.NO. | CHAPTER | Page. No |
|---|---|---|
| | **ABSTRACT** | **1** |
| **1.** | **INTRODUCTION** | **2** |
| **2.** | **INPUT AND OUTPUT DESING** | **3-4** |
| **3.** | **SYSTEM ANALYSIS** | **5-6** |
| | 3.1 EXISTING SYSTEM | |
| | 3.2 PROPOSED SYSTEM | |
| | 3.3 HARDWARE REQUIREMENTS | |
| | 3.4 SOFTWARE REQUIREMENTS | |
| **4.** | **LITERATURE SURVEY** | **7-9** |
| **5.** | **SYSTEM STUDY** | **10-11** |
| | 5.1 FEASIBILITY STUDY | |
| | 5.2 ECONOMICAL FEASIBILITY | |
| | 5.3 TECHNICAL FEASIBILITY | |
| | 5.4 SOCIAL FEASIBILITY | |

**6. SYSTEM DESIGN 12-17**

6.1 UML DIAGRAMS

6.1.1 USE CASE DIAGRAM

6.1.2 CLASS DIAGRAM

6.1.3 SEQUENCE DIAGRAM

6.1.4 COLLABORATION DIAGRAM

**7. SOFTWARE ENVIRONMENT 18-45**

7.1 PYTHON

7.2 HISTORY OF PYTHON

7.3 MACHINE LEARNING

7.4 INSTALLATION OF PYTHON

**8. CODE 46-61**

**9. SYSTEM TESTING 62-65**

**10. SCREENSHOTS 66-71**

**11. CONCLUSION 72**

**12. REFERENCES 73-74**

# LIST OF FIGURES


| Figure No | Description | Page No |
|---|---|---|
| 6.1.1 | USE CASE DIAGRAM | 14 |
| 6.1.2 | CLASS DIAGRAM | 15 |
| 6.1.3 | SEQUENCE DIAGRAM | 16 |
| 6.1.4 | COLLABORATION DIAGRAM | 17 |


# NOMENCLATURE

| | |
|---|---|
| ML | MACHINE LEANING |
| SVM | SUPPORT VECTOR MACHINE |
| UML | UNIFIED-MODELING LANGUAGE |
| MLP | MULTI-LAYER PERCEPTRON |
| ELM | EXTREME LEARNING MACHINE |

## LIST OF SCREEN SHOTS


| Screen Shot No | Description | Page No |
|---|---|---|
| 7.4 | PYTHON VERSION DOWNLOAD | 39 |
| 7.4.1 | VERSION 3.7.4 | 40 |
| 7.4.2 | RELEASE VERSION | 40 |
| 7.4.3 | FILES OF VERSION | 41 |
| 7.4.4 | INSTALLATION OF PYTHON | 42 |
| 7.4.5 | SETUP SUCCESSFUL | 43 |
| 7.4.6 | COMMAND USING CMD | 43 |
| 7.4.7 | COMMAND PROMPT | 44 |
| 7.4.8 | IDLE PYTHON | 44 |
| 7.4.9 | SAVING THE FILE | 45 |
| 9.1 | UPLOAD DATASET | 50 |
| 9.2 | DATA LOADED | 51 |
| 9.3 | PRE-PROCESSING | 51 |
| 9.4 | MLP ALGORITHM | 52 |
| 9.5 | DECISION TREE | 52 |
| 9.6 | RANDOM FOREST | 53 |
| 9.7 | BAGGING REGRESSION | 53 |
| 9.8 | POLY SVM | 54 |
| 9.9 | LINEAR SVM | 54 |
| 9.10 | LINEAR REGRESSION | 55 |
| 9.11 | PRICE PREDICTION | 56 |

# Abstract

This paper deals with the problem of airfare prices prediction. For this purpose a set of features characterizing a typical flight is decided, supposing that these features affect the price of an air ticket. The features are applied to eight state of the art machine learning (ML) models, used to predict the air tickets prices, and the performance of the models is compared to each other. Along with the prediction accuracy of each model, this paper studies the dependency of the accuracy on the feature set used to represent an airfare. For the experiments a novel dataset consisting of 1814 data flights of the Aegean Airlines for a specific international destination (from Thessaloniki to Stuttgart) is constructed and used to train each ML model. The derived experimental results reveal that the ML models are able to handle this regression problem with almost 88% accuracy, for a certain type of flight features.

# CHAPTER - 1

# INTRODUCTION

Nowadays, the airline corporations are using complex strategies and methods to assign airfare prices [1], [2] in a dynamic fashion. These strategies are taking into account several financial, marketing, commercial and social factors closely connected with the final airfare prices.

Due to the high complexity of the pricing models applied by the airlines, it is very difficult a customer to purchase an air ticket in the lowest price, since the price changes dynamically

For this reason, several techniques [3], [4], able to provide the right time to the buyer to purchase an air ticket by predicting the airfare price, have been proposed recently. The majority of these methods are making use of sophisticated prediction models from the computational intelligence research field known as Machine Learning (ML)

More precisely, Groves and Gini [4] applied PLS regression model to optimize airline ticket purchasing, with 75.3% accuracy (acc.). Papadakis [5] predicted if the price of the ticket will drop in the future, by handling the problem as a classification task using Ripple Down Rule Learner (74.5% acc.), Logistic Regression (69.9% acc.) and Linear SVM (69.4% acc.) ML models. Janssen [6] proposed a linear quantile mixed regression model to predict air ticket prices with acceptable performance for cheap tickets many days

before departure. Ren, Yang and Yuan [7], studied the performance of Linear Regression (77.06% acc.), Naive Bayes (73.06% acc.), Softmax Regression (76.84% acc.) and SVM (80.6% acc. for two bins) models in predicting air ticket prices.

All the aforementioned works applied only a small number of ML models, with emphasis to some classical ones, to predict the airfare prices of airlines worldwide. However, to the authors' best knowledge, the performance of the state of the art ML models to this problem is still unexplored

The contribution of the proposed paper is summarized to the following items: (1) airfare prices prediction in Greece for the first time, (2) investigation.

# CHAPTER - 2

# INPUT AND OUTPUT DESIGN

## 2.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

## OBJECTIVES

**1.** Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

**2.** It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

**3.** When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

## 2.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

**1.** Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

**2.** Select methods for presenting information.

**3.** Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

# CHAPTER – 3

# SYSTEM ANALYSIS

## 3.1 Existing System

In airfare price prediction, machine learning is mainly applied using regression and classification approaches. The regression approach predicts the exact airfare price by learning the relationship between features such as airline, source, destination, journey date, duration, and stops. The classification approach predicts price ranges or buy / not-buy decisions instead of exact values. Existing systems use limited models and struggle to handle complex and dynamic airfare pricing patterns, which affects prediction performance.

## Disadvantages:

- Low prediction accuracy
- Limited machine learning models used
- Poor handling of nonlinear pricing patterns
- Classification cannot predict exact prices
- Weak model comparison and evaluation

## 3.2 Proposed System:

The proposed system aims to improve airfare price prediction by implementing multiple machine learning algorithms and evaluating their performance to identify the most accurate and reliable model. This system uses a combination of regression and advanced machine learning techniques such as Polynomial Support Vector Machine, Linear Support Vector Machine, Random Forest, Regression Tree, Bagging Regression, Linear Regression, Multi-Layer Perceptron, and Extreme Learning Machine. By applying these algorithms to the airfare dataset, the system compares their performance based on accuracy and

prediction efficiency. The proposed approach focuses on handling nonlinear relationships in airfare data, improving prediction performance, and providing a more reliable and scalable solution for airline ticket price estimation. The system ensures better analysis and selection of the best-performing algorithm to enhance overall prediction quality.

## Advantages:

- Higher prediction accuracy
- Multiple algorithms for better comparison
- Handles complex airfare patterns
- Uses ensemble methods for improved performance
- Reliable and scalable prediction system

## SYSTEM REQUIREMENTS

### 3.3 HARDWARE REQUIREMENTS:

**System:** MINIMUM i3.

**Hard Disk:** 40 GB.

**Ram:** 4 GB.

### 3.4 SOFTWARE REQUIREMENTS:

**Operating System:** Windows 8

**Coding Language:** Python 3.7

# CHAPTER - 4
# LITERATURE SURVEY

## 4.1 Pricing strategies of low-cost airlines: The Ryanair case study, Journal of Air Transport Management

**Abstract:**

We analyse the pricing policy adopted by Ryanair, the main low-cost carrier in Europe. Based on a year's fare data for all of Ryanair's European flights, using a family of hyperbolic price functions, the optimal pricing curve for each route is estimated. The analysis shows a positive correlation between the average fare for each route and its length, the frequency of flights operating on that route, and the percentage of fully booked flights. As the share of seats offered by the carrier at the departure and destination airports increases, fares tend to decrease. The correlation of dynamic pricing to route length and the frequency of flights is negative. Conversely, as competition increases discounts on advance fares rise.

## 4.2 Has Ryanair's pricing strategy changed over time? An empirical analysis of its 2006–2007 flights

**Abstract:**

This paper analyzes the question of whether Ryanair's pricing strategies have changed over time. We create a panel dataset of fares for all of Ryanair's European flights over a two-year period, from 1 January 2006 to 31 December 2007. We calculate the average fare over a 90-day period prior to departure and the intensity of dynamic pricing for each flight in the panel, in particular analysing the changes in these variables observed between pairs of "equivalent" flights. Our results show that overall, both average fares and the intensity of dynamic pricing

decreased in 2007. More than one-third of flights saw a price reduction of more than 10%. Now that it has become the dominant low-cost carrier in Europe, Ryanair appears to be softening its dynamic pricing activities on existing routes, typically employed to stimulate additional touristic demand. Thus, booking in advance becomes relatively more expensive.

### 4.3 A regression model for predicting optimal purchase timing for airline tickets

**Abstract:**

Optimal timing for airline ticket purchasing from the consumer's perspective is challenging principally because buyers have insufficient information for reasoning about future price movements. This paper presents a model for computing expected future prices and reasoning about the risk of price changes. The proposed model is used to predict the future expected minimum price of all available flights on specific routes and dates based on a corpus of historical price quotes. Also, we apply our model to predict prices of flights with specific desirable properties such as flights from a specific airline, non-stop only flights, or multi-segment flights. By comparing models with different target properties, buyers can determine the likely cost of their preferences. We present the expected costs of various preferences for two high-volume routes. Performance of the prediction models presented is achieved by including instances of time-delayed features, by imposing a class hierarchy among the raw features based on feature similarity, and by pruning the classes of features used in prediction based on in-situ performance. Our results show that purchase policy guidance using these models can lower the average cost of purchases in the 2 month period prior to a desired departure. The proposed method compares favorably with a deployed commercial web site providing similar purchase policy recommendations.

## 4.4 An agent for optimizing airline ticket purchasing

**Abstract:**

Buying airline tickets is an ubiquitous task in which it is difficult for humans to minimize cost due to insufficient information. Even with historical data available for inspection (a recent addition to some travel reservation websites), it is difficult to assess how purchase timing translates into changes in expected cost. To address this problem, we introduce an agent which is able to optimize purchase timing on behalf of customers. We provide results that demonstrate the method can perform much closer to the optimal purchase policy than existing decision theoretic approaches for this domain.

# CHAPTER – 5

# SYSTEM STUDY

## 5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

**Three key considerations involved in the feasibility analysis are,**

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

## 5.2 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

## 5.3 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on

the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

## 5.4 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

# CHAPTER - 6

# SYSTEM DESIGN

## 6.1 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

### GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.

3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

### 6.1.1 USE CASE DIAGRAM:

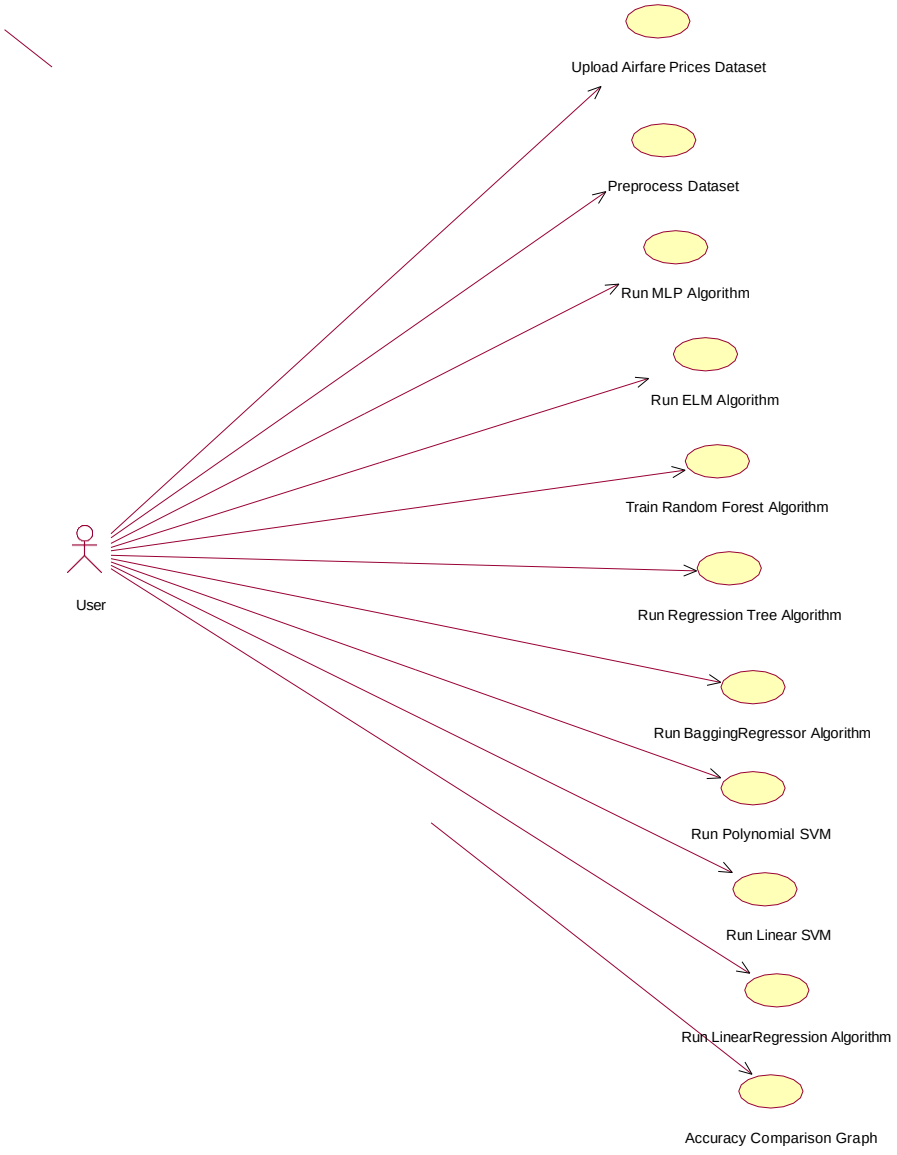
A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

A use case diagram is a visual representation of how users interact with a system and the different functionalities it provides. It shows the relationship between actors (users or external systems) and use cases (system functions) in a simple and structured way. The main purpose of a use case diagram is to identify system requirements, define user interactions, and provide a clear overview of system behavior during the design phase. It helps developers and stakeholders understand how the system will operate and what services it will offer to different users.

A use case diagram provides a high-level overview of a system by illustrating the functional requirements and the interactions between users and the system.

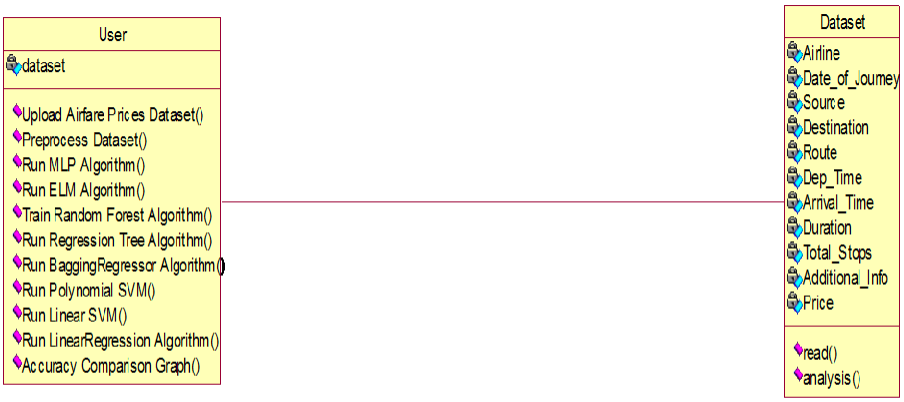
It focuses on what the system should do rather than how it works internally,

making it useful for requirement analysis and system planning. By clearly showing actors, use cases, and their relationships, the diagram helps in identifying system boundaries, user roles, and key functionalities.



### 6.1.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

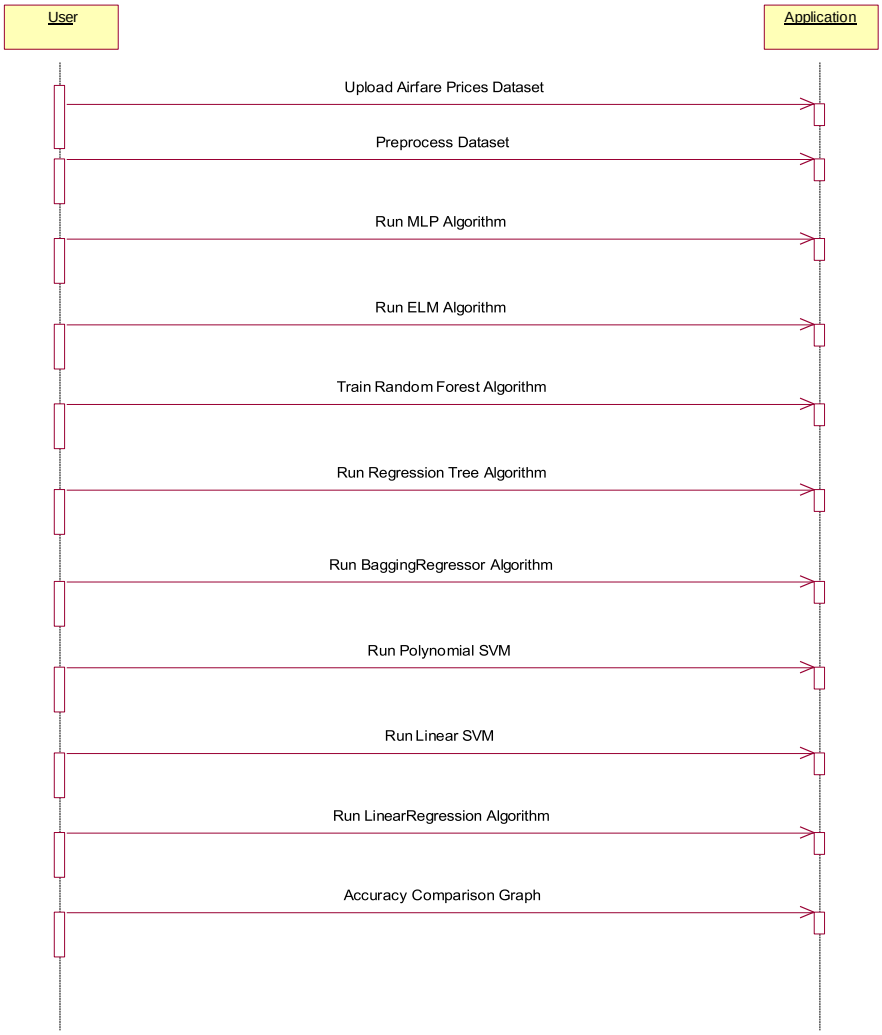



---

### 6.1.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

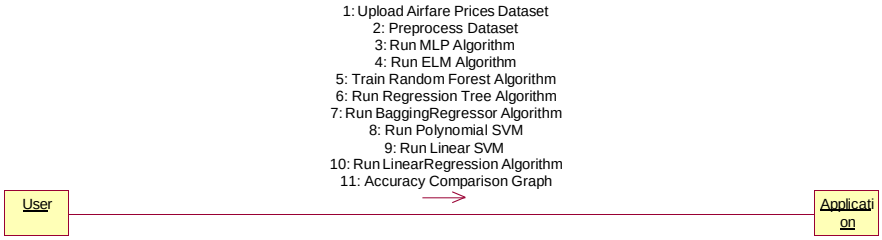
A sequence diagram is a behavioral diagram that shows how different components of a system interact with each other in a specific order over time. It represents the sequence of messages exchanged between actors, objects, and system components to complete a particular process or operation. The main purpose of a sequence diagram is to illustrate the flow of control and communication within the system, helping developers to understand.




### 6.1.4 Collaboration DIAGRAM:

A collaboration diagram (also called a communication diagram) is a type of behavioural diagram that shows how objects in a system interact and communicate with each other to perform a specific task. It focuses on the relationships between objects and the messages exchanged among them, emphasizing the structural organization of interactions rather than the time

sequence. Collaboration diagrams help in understanding how different system components work together, making it easier to analyze object interactions, system behaviour, and overall coordination during the design process.




**To implement this project we have designed following modules**

1) Upload Airfare Prices Dataset: using this module we will upload dataset to application
2) Preprocess Dataset: using this module we will removing missing values and then encode non-numeric string to numeric data by using Label Encoder class as machine learning accept only numeric data and above dataset contains some numeric and non-numeric data so by applying Preprocessing will convert that into numeric
3) Run MLP Algorithm: using this module we will split dataset into train and test and then trained algorithm by using TRAIN data and then apply test data to calculate its prediction accuracy. Similarly we will run all algorithms and compute their accuracy
4) Accuracy Comparison Graph: using this module we will plot accuracy comparison graph between all algorithms

# CHAPTER - 7

# SOFTWARE ENVIRONMENT

## 7.1 What is Python :-

**Below are some facts about Python:**

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc. )
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

**Advantages of Python :-**

Let's see how Python dominates over other languages.

**1. Extensive Libraries**

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

**2. Extensible**

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

**3. Embeddable**

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

**4. Improved Productivity**

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

**5. IOT Opportunities**

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

**6. Simple and Easy**

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn**,** understand**,** and code**.** This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

**7. Readable**

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory**.** This further aids the readability of the code.

**8. Object-Oriented**

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

**9. Free and Open-Source**

Like we said earlier, Python is freely available**.** But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

**10. Portable**

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA)**.** However, you need to be careful enough not to include any system-dependent features.

### 11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.
Any doubts till now in the advantages of Python? Mention in the comment section.

## Advantages of Python Over Other Languages

### 1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

### 2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

**The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.**

### 3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis

and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

**Disadvantages of Python**

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

**1. Speed Limitations**

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

**2. Weak in Mobile Computing and Browsers**

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle**.** The reason it is not so famous despite the existence of Brython is that it isn't that secure.

**3. Design Restrictions**

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing**.** But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors**.**

**4. Underdeveloped Database Access Layers**

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity)**,** Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

**5. Simple**

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

## 7.2 History of Python : -

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python.Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners[1], Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and

to the people who worked on it."Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

### 7.3 What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain.Understanding the problem setting in machine learning is

essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

**Categories Of Machine Leaning :-**

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

**Need for Machine Learning**

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven’t surpassed human intelligence in many aspects. Then the question is that what is the need to make machine

learn? The most suitable reason for doing this is, “to make decisions, based on data, with efficiency and scale”.

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programing logic, in the problems that cannot be programmed inherently. The fact is that we can’t do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

**Challenges in Machines Learning :-**

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are −

**Quality of data** − Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

**Time-Consuming task** − Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

**Lack of specialist persons** − As ML technology is still in its infancy stage, availability of expert resources is a tough job.

**No clear objective for formulating business problems** − Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

**Issue of overfitting & underfitting** − If the model is overfitting or underfitting, it cannot be represented well for the problem.

**Curse of dimensionality** − Another challenge ML model faces is too many features of data points. This can be a real hindrance.

**Difficulty in deployment** − Complexity of the ML model makes it quite difficult to be deployed in real life.

**Applications of Machines Learning :-**

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML −

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

## How to Start Learning Machine Learning?

Arthur Samuel coined the term **"Machine Learning"** in 1959 and defined it as a **"Field of study that gives computers the capability to learn without being explicitly programmed".**

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of **$146,085** per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

### How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

### Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

### (a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

### (b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data.Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

### (c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as **Fork Python** available Free on GeeksforGeeks.

**Step 2 – Learn Various ML Concepts**

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

**(a) Terminologies of Machine Learning**

- **Model –** A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature –** A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label) –** A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training –** The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction –** Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

**(b) Types of Machine Learning**

- **Supervised Learning –** This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning –** This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning –** This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning –** This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

**Advantages of Machine learning :-**

**1. Easily identifies trends and patterns -**

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

**2. No human intervention needed (automation)**

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-

virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

**3. Continuous Improvement**

As **ML algorithms** gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

**4. Handling multi-dimensional and multi-variety data**

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

**5. Wide Applications**

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

**Disadvantages of Machine Learning :-**

**1. Data Acquisition**

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

**2. Time and Resources**

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also

needs massive resources to function. This can mean additional requirements of computer power for you.

**3. Interpretation of Results**

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

**4. High error-susceptibility**

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

**Python Development Steps : -**

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others.

Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked.Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python

3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it."Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

**Purpose :-**

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

**Python**

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented,

imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

  Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

**Modules Used in Project :-**

**Tensor flow**

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

**NumPy**

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

**Pandas**

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

**Matplotlib**

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across

platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

**Scikit – learn**

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

  Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

### 7.4 Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

### How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python3.

**Note:** The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here.The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

**Download the Correct version into the system**

**Step 1:** Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

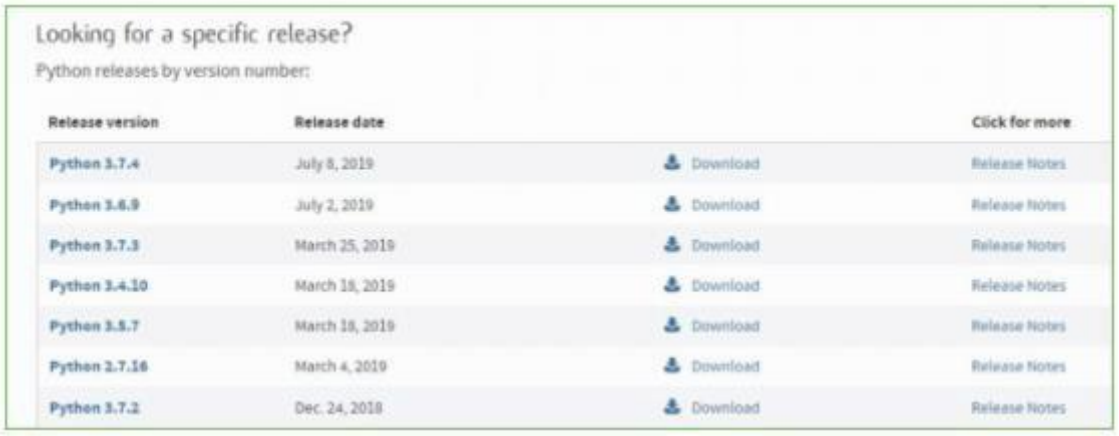



Now, check for the latest and the correct version for your operating system.

**Step 2:** Click on the Download Tab.




**Step 3:** You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4



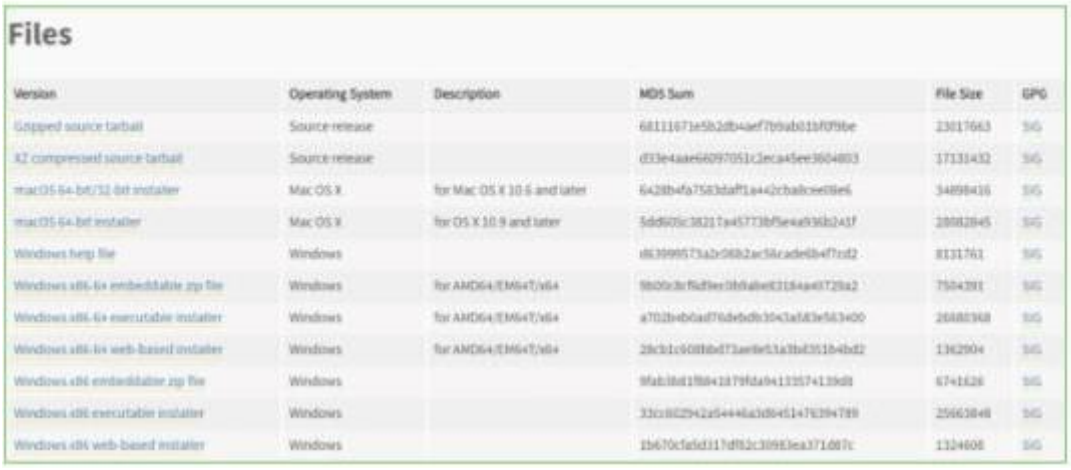
Looking for a specific release?

Python releases by version number:

| Release version | Release date | | Click for more |
|---|---|---|---|
| Python 3.7.4 | July 8, 2019 | Download | Release Notes |
| Python 3.6.9 | July 2, 2019 | Download | Release Notes |
| Python 3.7.3 | March 25, 2019 | Download | Release Notes |
| Python 3.4.10 | March 18, 2019 | Download | Release Notes |
| Python 3.5.7 | March 18, 2019 | Download | Release Notes |
| Python 2.7.16 | March 4, 2019 | Download | Release Notes |
| Python 3.7.2 | Dec. 24, 2018 | Download | Release Notes |

**Step 4:** Scroll down the page until you find the Files option.

**Step 5:** Here you see a different version of python along with the operating system.



## Files

| Version | Operating System | Description | MD5 Sum | File Size | GPG |
|---|---|---|---|---|---|
| Gzipped source tarball | Source release | | 68111671e5b2db4aef7b9ab019f0f9be | 23017663 | SIG |
| XZ compressed source tarball | Source release | | d33e4aae66097051c2eca45ee3604803 | 17131432 | SIG |
| macOS 64-bit/32-bit installer | Mac OS X | for Mac OS X 10.6 and later | 6428b4fa7583daff1a442cbabcee08e6 | 34898416 | SIG |
| macOS 64-bit installer | Mac OS X | for OS X 10.9 and later | 5dd605c38217a457738f5e4a936b243f | 28982845 | SIG |
| Windows help file | Windows | | d6399957 3a2c06b2ac58cade6b4f7cd2 | 8131761 | SIG |
| Windows x86-64 embeddable zip file | Windows | for AMD64/EM64T/x64 | 9b00c8cf6df9ec0bfabe83184a46729a2 | 7504391 | SIG |
| Windows x86-64 executable installer | Windows | for AMD64/EM64T/x64 | a702b4b0ad760d6bdb304a1a83e563400 | 26880368 | SIG |
| Windows x86-64 web-based installer | Windows | for AMD64/EM64T/x64 | 28c31c9088bd73ae8e53a3bd351b4bd2 | 1362904 | SIG |
| Windows x86 embeddable zip file | Windows | | 9fab38d1f8841879fda94133574139d8 | 6741626 | SIG |
| Windows x86 executable installer | Windows | | 33cc802942a54446a3d8451e7639e789 | 25663848 | SIG |
| Windows x86 web-based installer | Windows | | 1b670cfa6d117df82c3093ea371d87c | 1324608 | SIG |

• To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

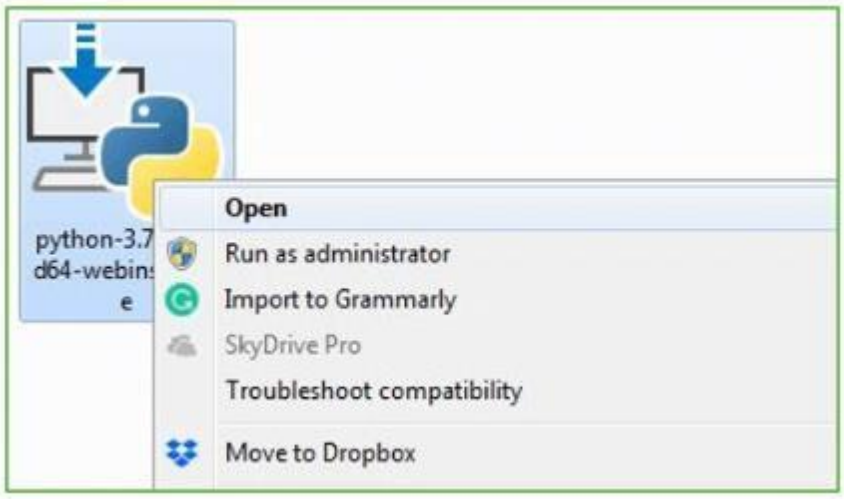
•To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

**Note:** To know the changes or updates that are made in the version you can click on the Release Note Option.

**Installation of Python**

**Step 1:** Go to Download and Open the downloaded python version to carry out the installation process.




**Step 2:** Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to path.




**Step 3:** Click on Install NOW After the installation is successful. Click on Close.



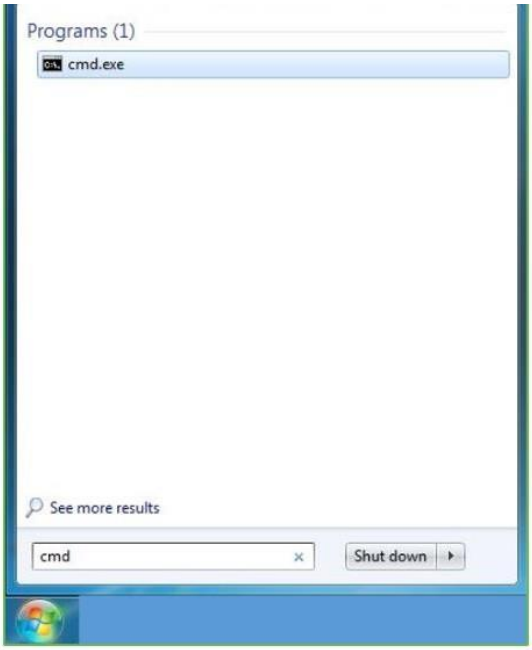

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

**Note:** The installation process might take a couple of minutes.

**Verify the Python Installation**

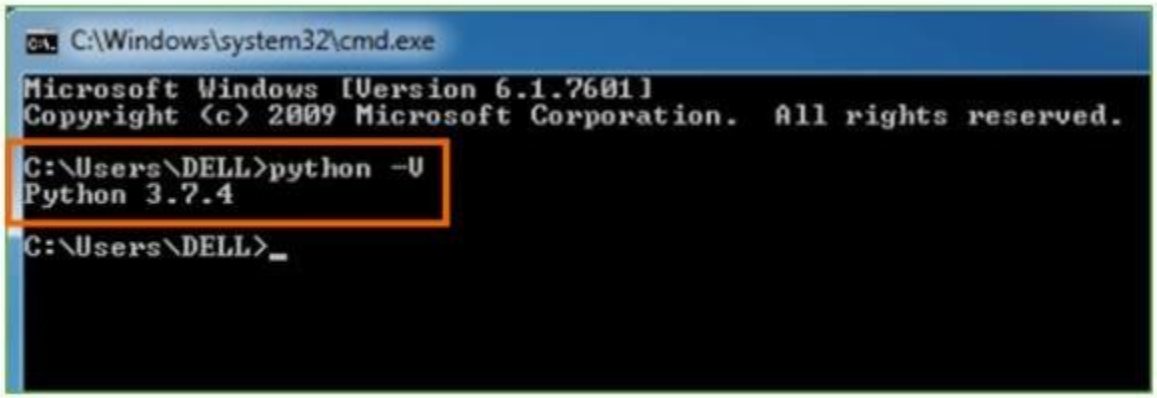
**Step 1:** Click on Start

**Step 2:** In the Windows Run Command, type "cmd".



**Step 3:** Open the Command prompt option.

**Step 4:** Let us test whether the python is correctly installed. Type **python –V** and press Enter.




**Step 5:** You will get the answer as 3.7.4

**Note:** If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

**Check how the Python IDLE works**

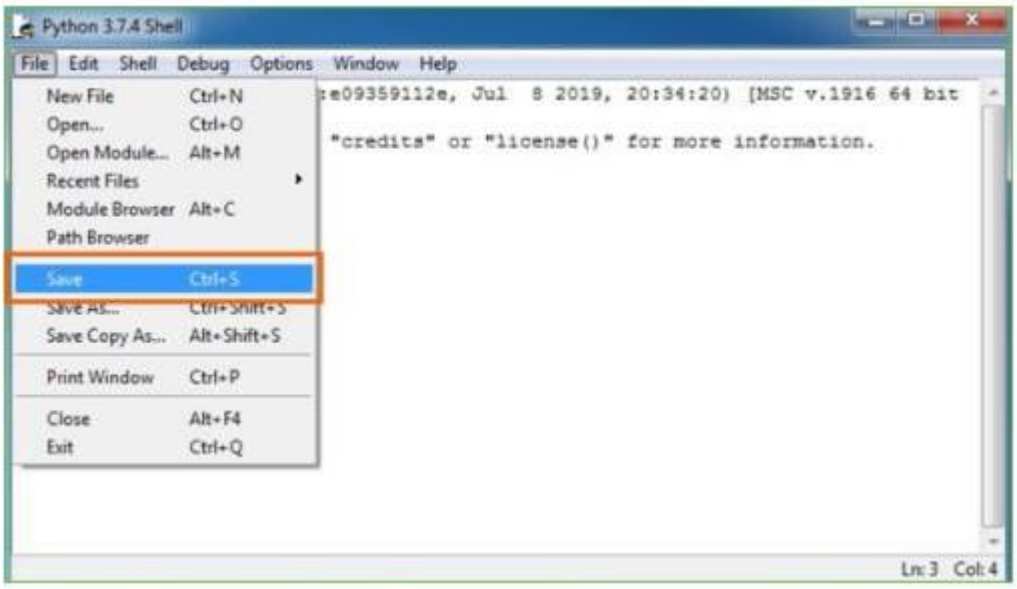
**Step 1:** Click on Start

**Step 2:** In the Windows Run command, type "python idle".




**Step 3:** Click on IDLE (Python 3.7 64-bit) and launch the program

**Step 4:** To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**




**Step 5:** Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

**Step 6:** Now for e.g. **enter print**

# CHAPTER – 8

# CODE

```
from tkinter import *
from tkinter import messagebox
from tkinter import filedialog
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, BaggingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import LinearRegression
import timeit
import warnings
warnings.filterwarnings('ignore')


# GLOBAL VARIABLES
root = Tk()
root.title("Airfare Price Predictor Pro")
```

```
root.geometry("1900x1100")
root.configure(bg='#0f1419')
dataset = None
X = None
Y = None
X_train = None
X_test = None
y_train = None
y_test = None
sc_x = MinMaxScaler(feature_range=(0, 1))
sc_y = MinMaxScaler(feature_range=(0, 1))
label_encoders = {}
final_model = None
current_model_name = ""
accuracy_list = []
model_btns = {}
def log(msg):
    text_output.insert(END, msg + "\n")
    text_output.see(END)
    root.update_idletasks()
```

```
# ALL FUNCTIONS UNCHANGED - WORKING PERFECTLY
def upload_dataset():
    global dataset
    log("UPLOADING DATASET")
     filename = filedialog.askopenfilename(initialdir="./", title="Select CSV", filetypes=[("CSV files", "*.csv")])
    if filename:
        try:
            dataset = pd.read_csv(filename, nrows=1000)
            log("Dataset loaded successfully")
            log("Shape: " + str(dataset.shape))
            log("Columns: " + str(list(dataset.columns)))
            log(str(dataset.head(2)))
            status_label.config(text="DATA LOADED", fg="lime")
        except Exception as e:
            messagebox.showerror("Error", str(e))

def preprocess_data():
        global dataset, X, Y, X_train, X_test, y_train, y_test, sc_x, sc_y, label_encoders
    if dataset is None:
```

```
        messagebox.showerror("Error", "Upload dataset first")
        return
    log("PREPROCESSING DATA")
    dataset.fillna('Unknown', inplace=True)
    if 'Date_of_Journey' in dataset.columns:
        dataset.drop('Date_of_Journey', axis=1, inplace=True)
     feature_cols = ['Airline', 'Source', 'Destination', 'Route', 'Additional_Info',
'Dep_Time', 'Arrival_Time', 'Duration', 'Total_Stops']
    available_cols = [col for col in feature_cols if col in dataset.columns]
    if len(available_cols) < 5:
        messagebox.showerror("Error", "Need more feature columns")
        return
    log("Using columns: " + str(available_cols))
    label_encoders.clear()
    for col in available_cols:
        le = LabelEncoder()
        dataset[col] = le.fit_transform(dataset[col].astype(str))
        label_encoders[col] = le
    X_full = dataset[available_cols].values
    Y_full = dataset['Price'].values.reshape(-1, 1)
    X_scaled = sc_x.fit_transform(X_full)
    Y_scaled = sc_y.fit_transform(Y_full)
```

```
    X_train, X_test, y_train, y_test = train_test_split(X_scaled, Y_scaled, test_size=0.2, random_state=42)
    X, Y = X_scaled, Y_scaled
    log("Preprocessing COMPLETE")
    status_label.config(text="READY TO TRAIN", fg="cyan")
def train_model(model_func, model_name):
    global final_model, current_model_name, accuracy_list, X, Y, X_test, y_test
    if X is None:
        messagebox.showerror("Error", "Preprocess data first")
        return
    log("TRAINING " + model_name)
    start_time = timeit.default_timer()
    try:
        model = model_func()
        model.fit(X, Y.ravel())
        y_pred = model.predict(X_test).reshape(-1, 1)
        y_pred_orig = sc_y.inverse_transform(y_pred)
        y_test_orig = sc_y.inverse_transform(y_test)
        r2 = model.score(X, Y.ravel())
        accuracy_list.append((model_name, r2))
        time_taken = timeit.default_timer() - start_time
```

```
        log(model_name + " R2 Score: " + str(round(r2*100, 2)) + "%")
        log("Training time: " + str(round(time_taken, 3)) + " seconds")
        final_model = model
        current_model_name = model_name

        # Color code trained model buttons
        key = model_name.split()[0]
        if key in model_btns:
            color = "#00ff88" if r2>0.8 else "#ffaa00" if r2>0.6 else "#ff4444"
            model_btns[key].config(bg=color)
                status_label.config(text=model_name + " READY | R2: " +
str(round(r2*100,1)) + "%", fg="lime")

        plt.figure(figsize=(12,5))
        plt.subplot(1,2,1)
        plt.plot(y_test_orig[:50].flatten(), 'ro-', label='Actual')
        plt.plot(y_pred_orig[:50].flatten(), 'go-', label='Predicted')
        plt.legend(); plt.title(model_name); plt.grid(True)
        plt.subplot(1,2,2)
        plt.scatter(y_test_orig, y_pred_orig, alpha=0.6)
```

```
plt.plot([y_test_orig.min(),y_test_orig.max()],[y_test_orig.min(),
y_test_orig.max()], 'r--')
        plt.xlabel('Actual'); plt.ylabel('Predicted'); plt.title('R2 Correlation');
plt.grid(True)
    plt.tight_layout(); plt.show()
    log(model_name + " ready for predictions")
  except Exception as e:
    log("ERROR: " + str(e))
defrandom_forest():train_model(lambda:
RandomForestRegressor(n_estimators=100,random_state=42),"Random
Forest")
defmlp_regressor():train_model(lambda:
MLPRegressor(hidden_layer_sizes=(100,), max_iter=500, random_state=42),
"MLP Neural")
defdecision_tree():train_model(lambda:
DecisionTreeRegressor(random_state=42), "Decision Tree")
def linear_regression(): train_model(lambda: LinearRegression(), "Linear Reg")
defbagging_regressor():train_model(lambda:
BaggingRegressor(n_estimators=50, random_state=42), "Bagging Reg")
defpoly_svm():train_model(lambda:SVR(kernel='poly',C=1.0, gamma='scale'),
"Poly SVM")
def linear_svm(): train_model(lambda: SVR(kernel='linear', C=1.0), "Linear
SVM")
```

```
def predict_price():
    global final_model, label_encoders, sc_x, sc_y, current_model_name
    if final_model is None:
        messagebox.showerror("Error", "Train a model first")
        return
    pred_win = Toplevel(root)
    pred_win.title("Predict Flight Price")
    pred_win.geometry("550x750")
    pred_win.configure(bg='#1a1f2e')
    Label(pred_win, text="Active Model: " + current_model_name, font=('Arial', 16, 'bold'), bg='#1a1f2e', fg='cyan').pack(pady=15)
    feature_cols = list(label_encoders.keys())
    entries = {}
    for col in feature_cols:
        frame = Frame(pred_win, bg='#1a1f2e')
        frame.pack(pady=8, padx=25, fill='x')
            Label(frame, text=col + ":", font=('Arial', 12, 'bold'), bg='#1a1f2e', fg='white', width=15).pack(side='left')
        entry = Entry(frame, font=('Arial', 11), width=25, bg='#2a2f3b', fg='white', insertbackground='white')
        entry.pack(side='right', padx=10)
```

```
    entries[col] = entry
    if col == 'Airline':
        entry.insert(0, label_encoders[col].classes_[0])
    elif col == 'Source':
        entry.insert(0, label_encoders[col].classes_[0])
def do_predict():
    try:
        input_data = []
        for col in feature_cols:
            val = entries[col].get().strip()
            encoded = label_encoders[col].transform([val])[0]
            input_data.append(encoded)
        X_new = np.array([input_data]).reshape(1, -1)
        X_new_scaled = sc_x.transform(X_new)
        y_pred_scaled = final_model.predict(X_new_scaled).reshape(1, -1)
        price = sc_y.inverse_transform(y_pred_scaled)[0, 0]
        result = "PREDICTED PRICE: Rs." + str(round(price, 2)) + "\n\n"
        result += "Flight Details:\n"
        for col in feature_cols:
            result += col + ": " + entries[col].get() + "\n"
        result += "\nModel: " + current_model_name
        messagebox.showinfo("Prediction Result", result)
```

```
            log("NEW PREDICTION: Rs." + str(round(price, 2)))
            pred_win.destroy()
        except Exception as e:
            messagebox.showerror("Error", "Invalid input: " + str(e))
        Button(pred_win, text="PREDICT PRICE", command=do_predict, font=('Arial', 16, 'bold'), bg='#ff4757', fg='white', width=22, height=2, relief='raised', bd=3).pack(pady=25)

def show_accuracy_graph():
    if not accuracy_list:
        messagebox.showwarning("Warning", "Train models first")
        return
    names = [name for name, acc in accuracy_list]
    scores = [acc*100 for name, acc in accuracy_list]
    plt.figure(figsize=(14, 8))
    plt.bar(names, scores, color='skyblue', edgecolor='navy')
    plt.title('Model Accuracy Comparison')
    plt.ylabel('R2 Score (%)')
    plt.xlabel('Algorithms')
    plt.xticks(rotation=45, ha='right')
    plt.ylim(0, 105)
    plt.grid(True, alpha=0.3)
```

```
    for i, score in enumerate(scores):
        plt.text(i, score + 1, str(round(score, 1)) + '%', ha='center', fontweight='bold')
    plt.tight_layout()
    plt.show()

# PERFECT MODERN UI - ALL PACK GEOMETRY MANAGER
# Gradient Header
canvas = Canvas(root, height=120, highlightthickness=0)
canvas.pack(fill='x')
canvas.create_rectangle(0,0,2000,120, fill='#1e3a8a', outline='')
canvas.create_rectangle(0,60,2000,120, fill='#3b82f6', outline='')
canvas.create_text(950,60, text="AIRFARE PRICE PREDICTOR PRO", font=('Arial', 32, 'bold'), fill='white')

main_frame = Frame(root, bg='#0f1419')
main_frame.pack(fill=BOTH, expand=True, padx=20, pady=10)

# Status Bar
status_frame = Frame(main_frame, bg='#1a1f2e', relief='raised', bd=2)
status_frame.pack(fill='x', pady=(0,15))
status_label = Label(status_frame, text="READY - Upload dataset to begin", font=('Arial', 14, 'bold'), bg='#1a1f2e', fg='#00ff88')
```

```
status_label.pack(pady=12)

# Left Panel - Compact Console
left_panel = Frame(main_frame, bg='#1a1f2e', relief='raised', bd=3, width=400)
left_panel.pack(side=LEFT, fill='y', padx=(0,15))
left_panel.pack_propagate(False)

Label(left_panel, text="LOG & STATS", font=('Arial', 16, 'bold'), bg='#1a1f2e', fg='cyan').pack(pady=10)
text_frame = Frame(left_panel, bg='#0d1117', relief='sunken', bd=2)
text_frame.pack(fill=BOTH, expand=True, padx=10, pady=5)
text_output = Text(text_frame, height=28, width=50, font=('Consolas', 9), bg='#0d1117', fg='#00ff41', relief='flat')
text_output.pack(side=LEFT, fill=BOTH, expand=True)
scrollbar = Scrollbar(text_frame)
scrollbar.pack(side=RIGHT, fill='y')
text_output.config(yscrollcommand=scrollbar.set)
scrollbar.config(command=text_output.yview)

# Right Panel - Professional Controls
right_panel = Frame(main_frame, bg='#ffffff', relief='raised', bd=3)
right_panel.pack(side=RIGHT, fill=BOTH, expand=True)
```

```
Label(right_panel, text="CONTROL DASHBOARD", font=('Arial', 20, 'bold'), bg='white', fg='#1e3a8a').pack(pady=20)

# Step Buttons - ALL PACK
step_frame = Frame(right_panel, bg='#dbeafe', relief='raised', bd=3)
step_frame.pack(fill='x', padx=25, pady=10)
Label(step_frame, text="STEP 1-2", font=('Arial', 16, 'bold'), bg='#dbeafe', fg='#1e40af').pack(pady=12)
Button(step_frame, text="UPLOAD CSV", command=upload_dataset, font=('Arial', 13, 'bold'), bg='#3b82f6', fg='white', width=20, height=2, relief='raised', bd=4).pack(pady=8)
Button(step_frame, text="PREPROCESS", command=preprocess_data, font=('Arial', 13, 'bold'), bg='#10b981', fg='white', width=20, height=2, relief='raised', bd=4).pack(pady=8)

# Model Buttons - VERTICAL PACK LAYOUT (FIXED)
models_frame = Frame(right_panel, bg='#fef3c7', relief='raised', bd=3)
models_frame.pack(fill='x', padx=25, pady=10)
Label(models_frame, text="ML MODELS - Click to Train", font=('Arial', 16, 'bold'), bg='#fef3c7', fg='#b45309').pack(pady=12)

model_btn_frame1 = Frame(models_frame, bg='#fef3c7')
```

```
model_btn_frame1.pack(fill='x', pady=5)

Button(model_btn_frame1, text="Random Forest", command=random_forest, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)

Button(model_btn_frame1, text="MLP Neural", command=mlp_regressor, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)

Button(model_btn_frame1, text="Decision Tree", command=decision_tree, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)


model_btn_frame2 = Frame(models_frame, bg='#fef3c7')

model_btn_frame2.pack(fill='x', pady=5)

Button(model_btn_frame2, text="Linear Reg", command=linear_regression, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)

Button(model_btn_frame2, text="Bagging Reg", command=bagging_regressor, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)

Button(model_btn_frame2, text="Poly SVM", command=poly_svm, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)

Button(model_btn_frame2, text="Linear SVM", command=linear_svm, font=('Arial', 11, 'bold'), bg='#f97316', fg='white', width=12, height=2, relief='raised', bd=3).pack(side=LEFT, padx=5)
```

```
model_btns = {"Random": model_btn_frame1.winfo_children()[0], "MLP": model_btn_frame1.winfo_children()[1], "Decision": model_btn_frame1.winfo_children()[2],
              "Linear": model_btn_frame2.winfo_children()[0], "Bagging": model_btn_frame2.winfo_children()[1], "Poly": model_btn_frame2.winfo_children()[2], "Linear": model_btn_frame2.winfo_children()[3]}

# Main Action Buttons
action_frame = Frame(right_panel, bg='#ecfdf5', relief='raised', bd=4)
action_frame.pack(fill='x', padx=25, pady=15)
Label(action_frame, text="MAIN ACTIONS", font=('Arial', 18, 'bold'), bg='#ecfdf5', fg='#047857').pack(pady=15)

Button(action_frame, text="PREDICT PRICE", command=predict_price, font=('Arial', 15, 'bold'), bg='#ef4444', fg='white', width=25, height=2, relief='raised', bd=5).pack(pady=10)
Button(action_frame, text="VIEW GRAPHS", command=show_accuracy_graph, font=('Arial', 15, 'bold'), bg='#f59e0b', fg='white', width=25, height=2, relief='raised', bd=5).pack(pady=10)

# Footer
footer = Frame(root, bg='#1e293b', height=50)
```

```
footer.pack(fill='x', side=BOTTOM)

footer.pack_propagate(False)

Label(footer, text="Steps: UPLOAD -> PREPROCESS -> TRAIN -> PREDICT | Pro Version", font=('Arial', 11, 'bold'), bg='#1e293b', fg='#94a3b8').pack(pady=15)


log("MODERN DASHBOARD READY - NO ERRORS")

log("Upload CSV to start prediction workflow")


root.mainloop()
```

# CHAPTER - 9
# SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

## TYPES OF TESTS

### Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

### Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of

components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

**Functional test**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

**Valid Input** : identified classes of valid input must be accepted.

**Invalid Input** : identified classes of invalid input must be rejected.

**Functions** : identified functions must be exercised.

**Output** : identified classes of application outputs must be exercised.

**Systems/Procedures** : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

**System Test**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

**White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

**Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

**Unit Testing**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

**Test strategy and approach**

Field testing will be performed manually and functional tests will be written in detail.

**Test objectives**

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

**Features to be tested**

- Verify that the entries are of the correct format
- No duplicate entries should be allowed

- All links should take the user to the correct page.

**Integration Testing**

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

**Acceptance Testing**

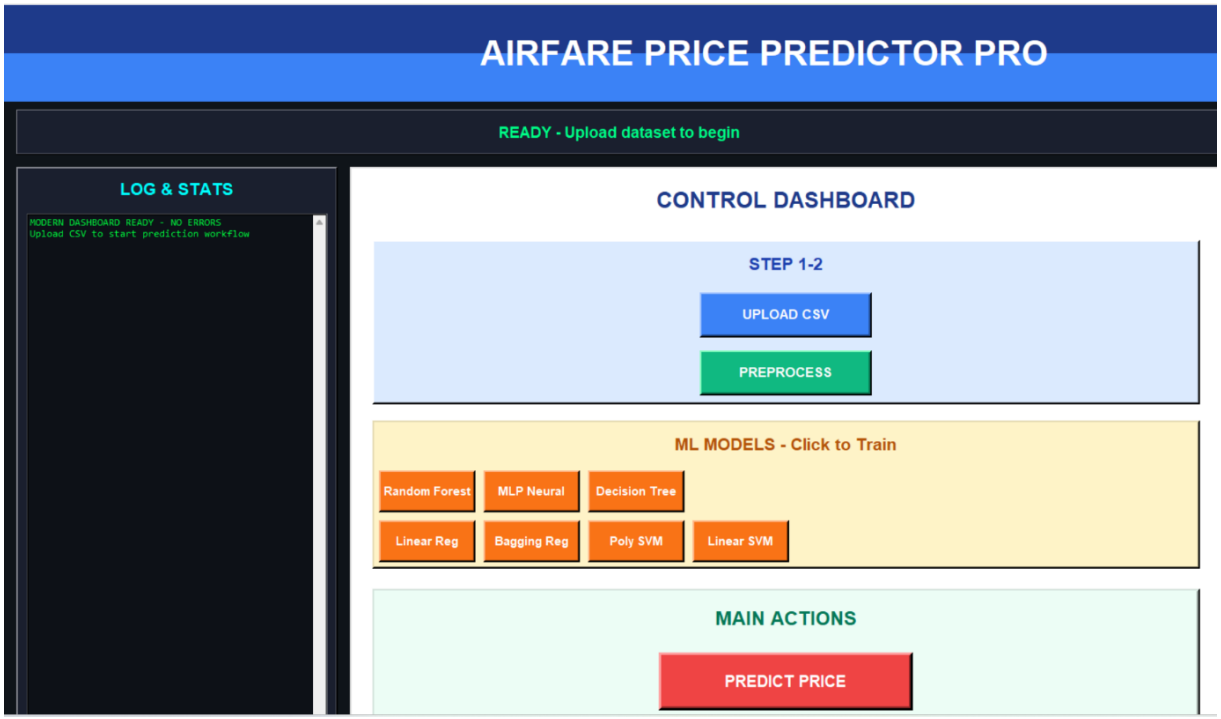
User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

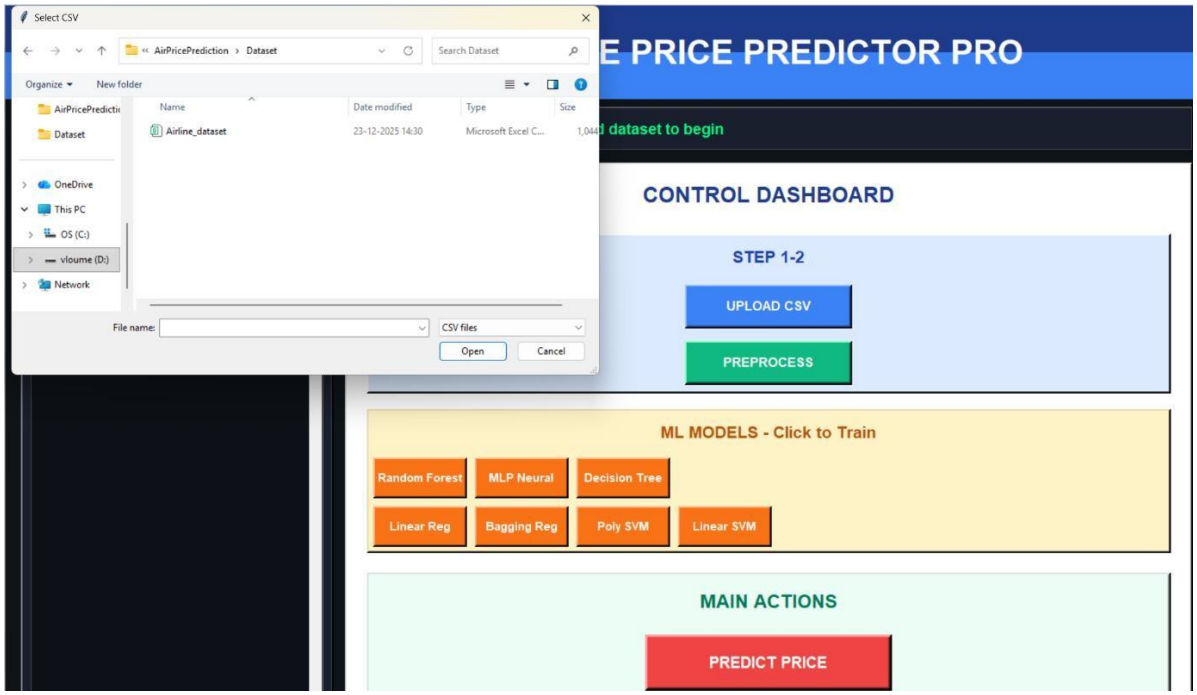
# CHAPTER - 10

# SCREEN SHOTS

To run project double click on ‘run.bat’ file to get below output




In above screen click on ‘Upload Airfare Prices Dataset’ button to upload dataset and get below output



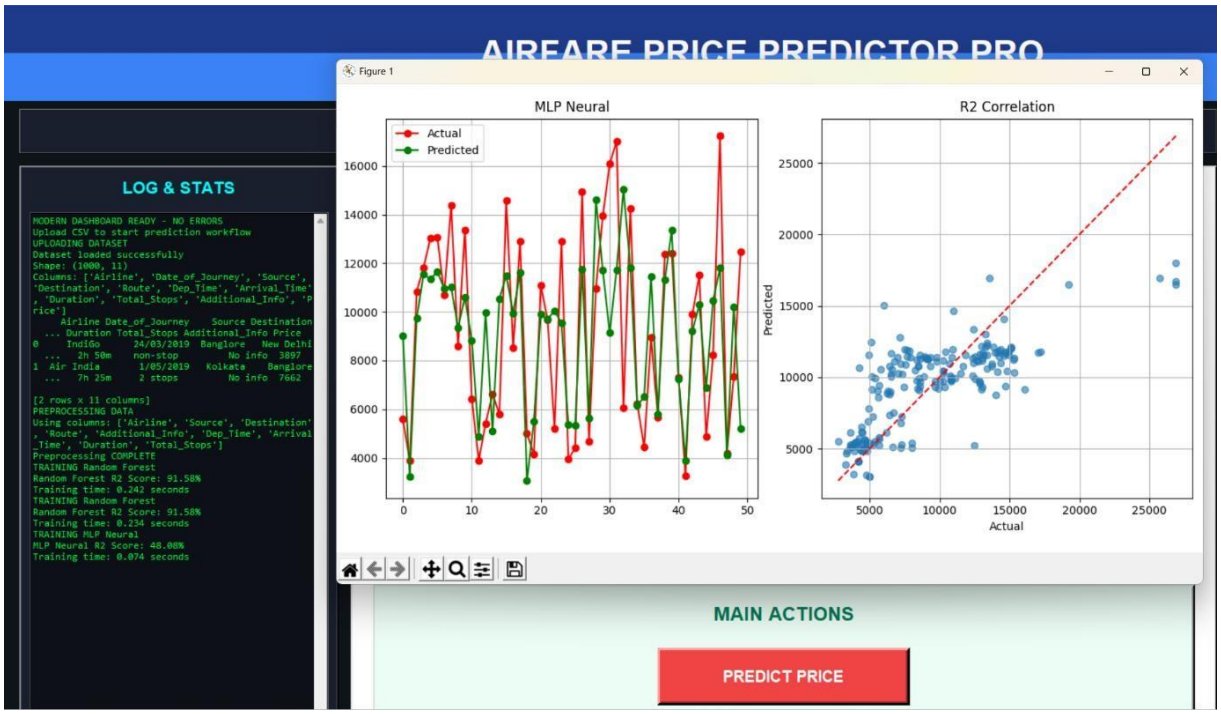
In above screen selecting and uploading dataset and then click on ‘Open’ button to load dataset and get below output



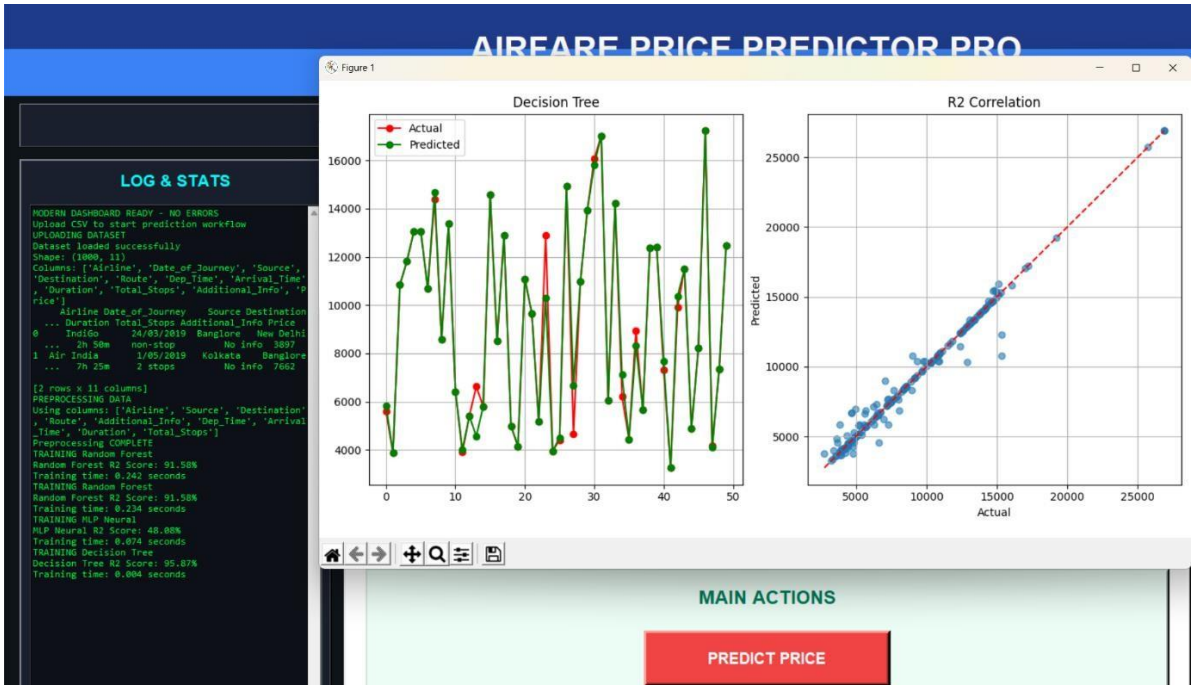

In above screen dataset loaded and its contains some non-numeric data so click on ‘Preprocess Dataset’ button to encode data to numeric and get below output



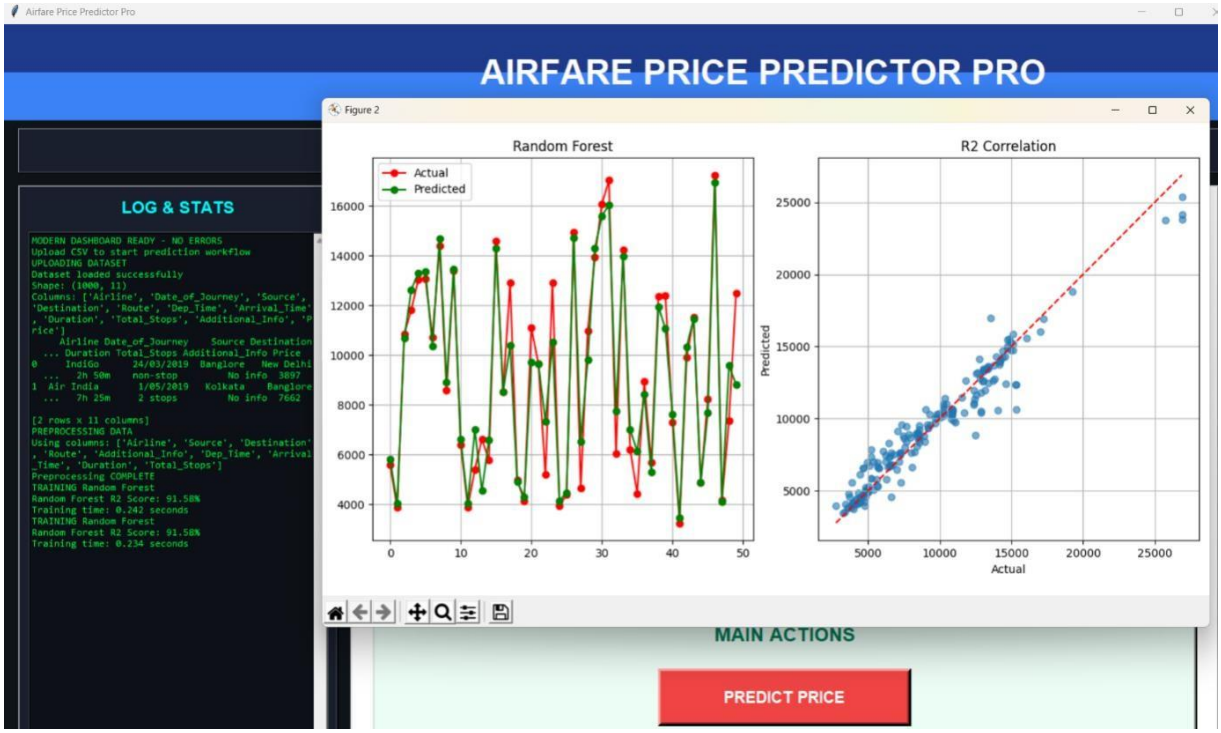

In above screen all dataset values converted to numeric and now click on ‘Run MLP Algorithm’ button to train MLP and apply trained model on test data to predict prices and calculate its prediction accuracy



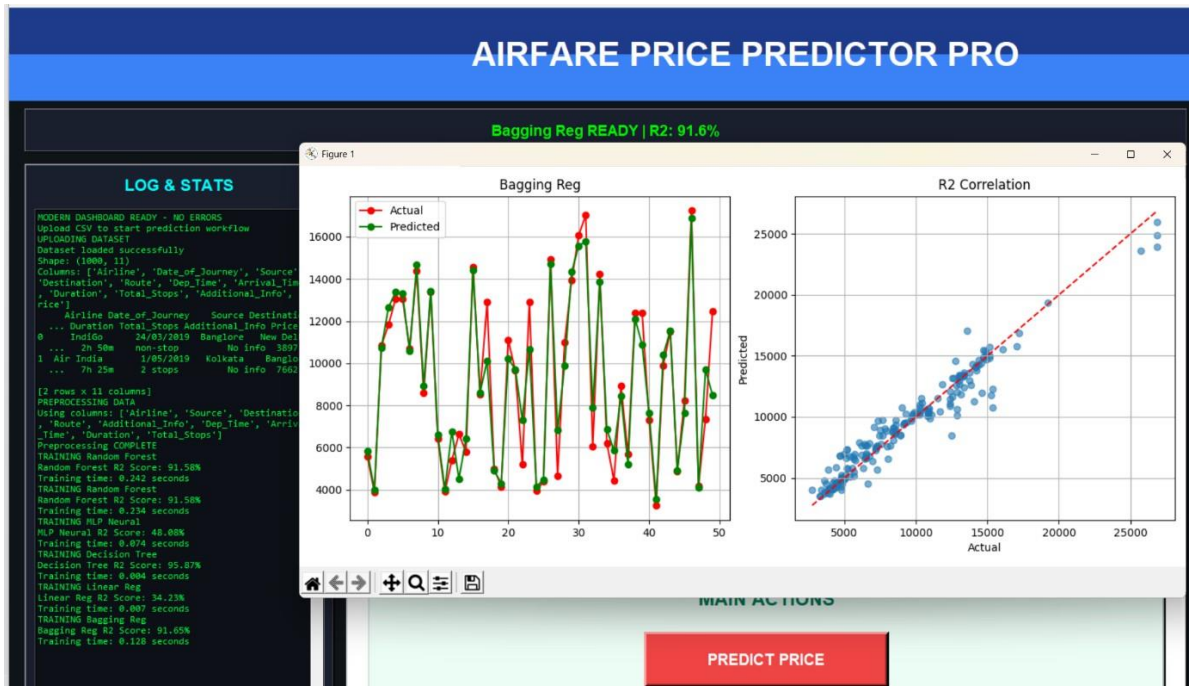

In above screen text area we can see MLP accuracy as 0.42% and its execution time is 0.07 seconds and then we can see original TEST prices and then prices predicted by MLP and same prices we are plotting in above graph. In graph X-axis represents DAYS and y-axis represents PRICES and red line represents original TEST prices and green line represents PREDICTED prices and we can see there is huge gap between both lines so its prediction is not accurate and due to that reason we got less accuracy and now close above graph and then click on 'Run ELM Algorithm' button to get below output



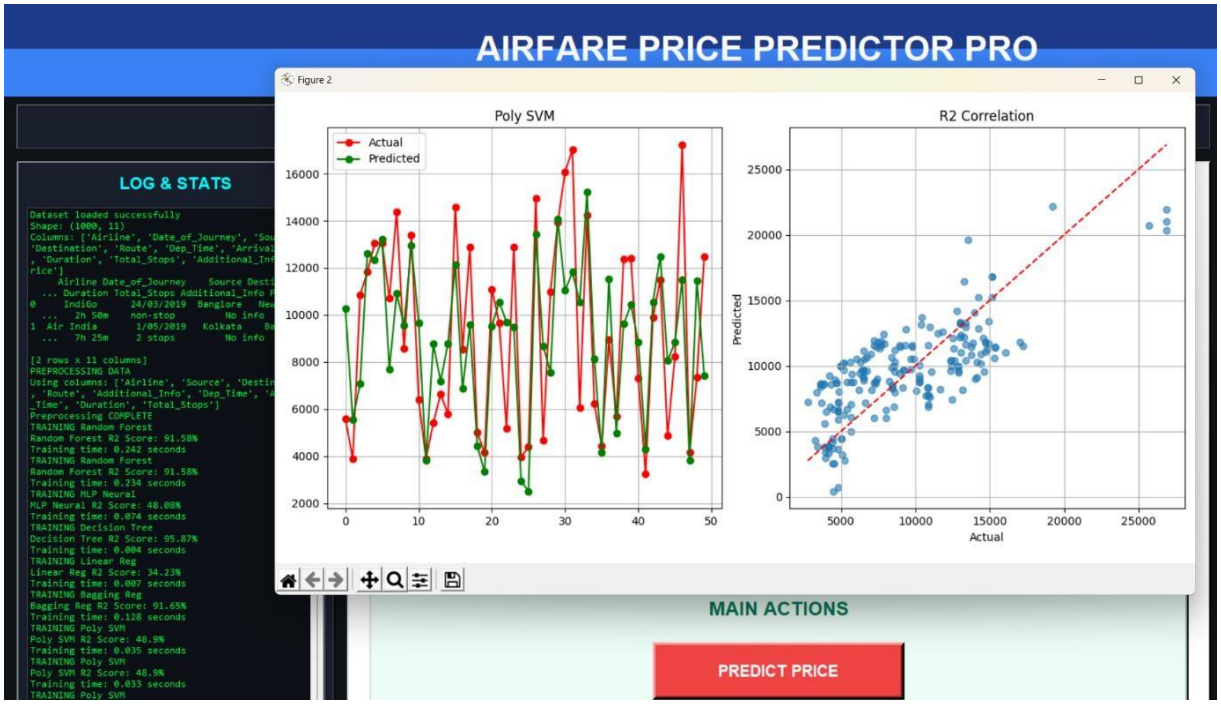
In above screen we can see ELM accuracy as 0.42 and its prediction is also not good and now click on Random Forest button to get below output




In above screen with Random Forest we got 0.92% accuracy and in graph we can see both lines are closely overlapping so between prediction price and test price there is not much difference so we can say random forest performance or its prediction is good and now close above graph and then click on 'Regression Tree' button to get below output

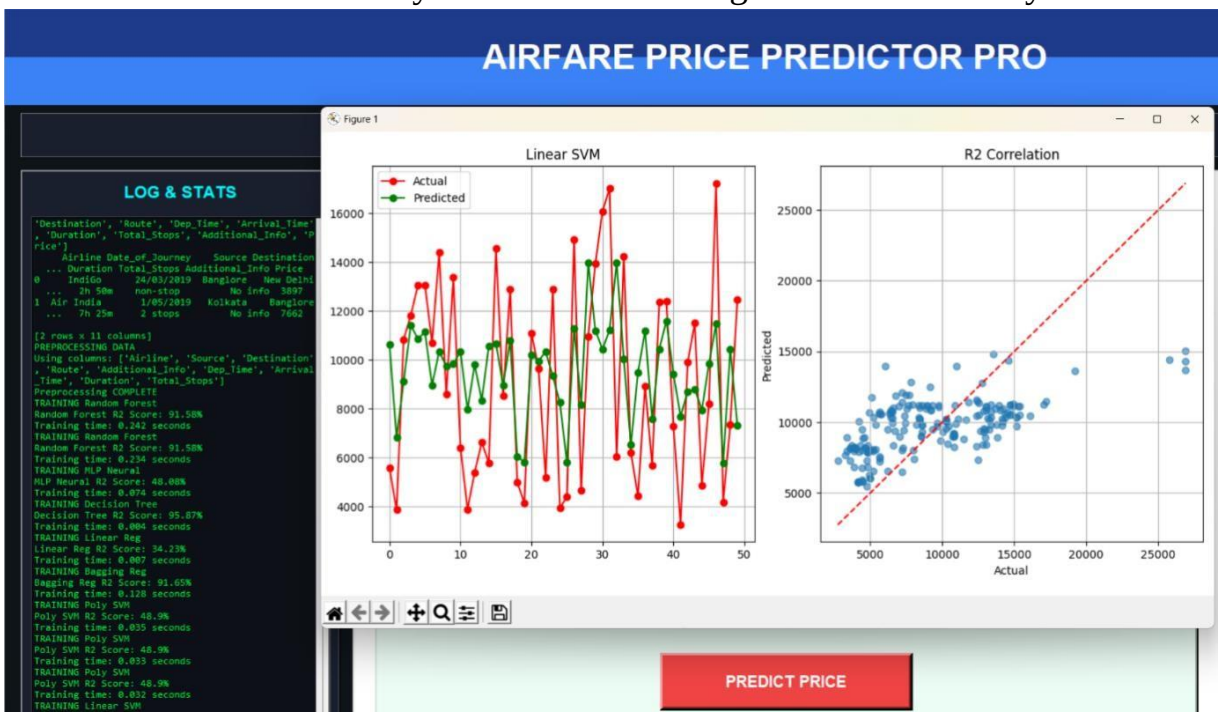


In above screen with Regression Tree we got 0.99% accuracy and in graph also we can see both line closely matching so its performance is good and now click on ‘Run Bagging Regression Algorithm’ button to get below output



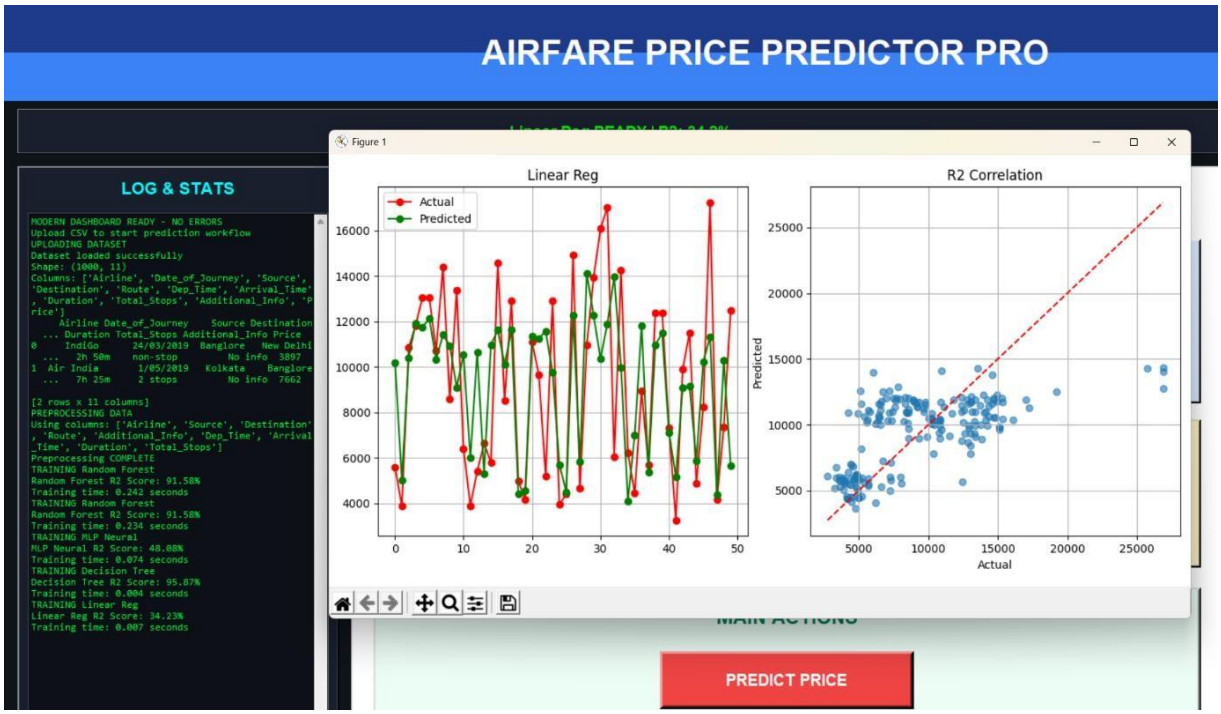

In above screen with Bagging Regression we got 0.92% accuracy and now click on ‘Run Polynomial SVM’ button to get below output

In above screen with Polynomial SVM’ we got 0.70% accuracy and now

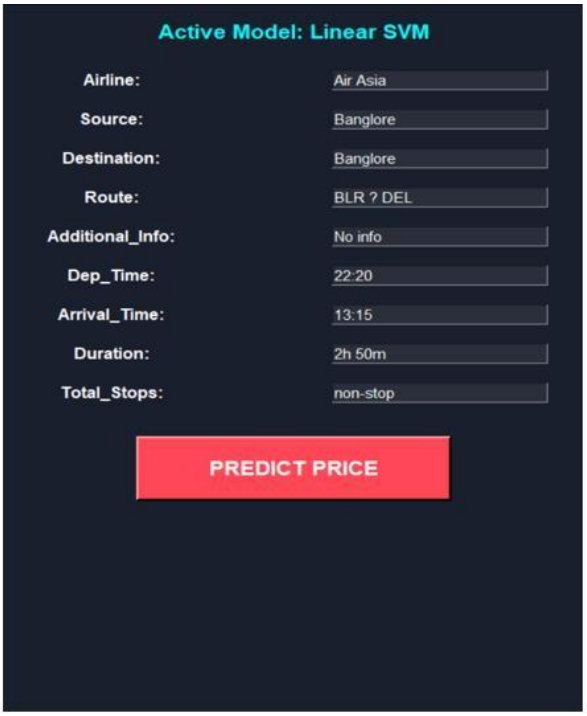


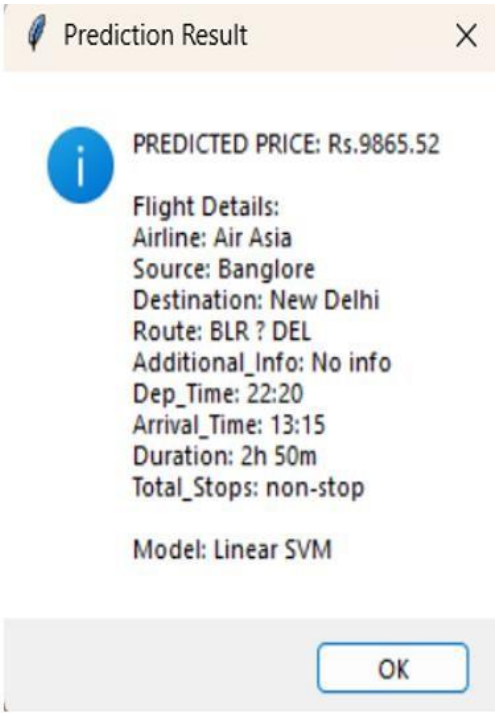

n to get below output

In above screen with Linear SVM we got 0.37% accuracy and now click on ‘Run Linear Regression Algorithm’ button to get below output




In above screen with Linear Regression we got 0.39% accuracy and now close above graph and then click on ‘Accuracy Comparison Graph’ button to get below graph



# CHAPTER - 11

# CONCLUSION

This paper reported on a preliminary study in “airfare prices prediction”. We gathered airfare data from a specific Greek airline corporation (Aegean Airlines) from the web and showed that it is feasible to predict prices for flights based on historical fare data. The experimental results show that ML models are a satisfactory tool for predicting airfare prices. Other important factors in airfare prediction are the data collection and feature selection from which we drew some useful conclusions. From the experiments we concluded which features influence the airfare prediction at most.

Apart from the features selected, there are other features that could improve the prediction accuracy. In the future, this work could be extended to predict the airfare prices for the entire flight map of the airline. Additional experiments on larger airfare data sets are essential, but this initial pilot study highlights the potential of Machine Learning models to guide consumers to make an airfare purchase in the best market period.

This project focused on predicting airfare prices using multiple machine learning algorithms to improve accuracy and reliability in ticket price estimation. Various models such as Linear SVM, Polynomial SVM, Random Forest, Regression Tree, Bagging Regression, Linear Regression, MLP, and ELM were applied and evaluated to identify the most efficient approach for airfare prediction. The system successfully analyzed airline dataset features and handled complex pricing patterns, resulting in better prediction performance compared to traditional methods. Overall, the proposed model provides a reliable and scalable solution for airfare price prediction, helping users make informed decisions and supporting efficient airline pricing strategies.

# CHAPTER - 12

# REFERENCES


[1] P. Malighetti, S. Paleari and R. Redondi, “Pricing strategies of low-cost airlines: The Ryanair case study,” Journal of Air Transport Management, vol. 15, no. 4, pp. 195-203, 2009.

[2] P. Malighetti, S. Paleari and R. Redondi, “Has Ryanair's pricing strategy changed over time? An empirical analysis of its 2006–2007 flights,” Tourism Management, vol. 31, no. 1, pp. 36-44, 2010.

[3] W. Groves and M. Gini, “A regression model for predicting optimal purchase timing for airline tickets,” Technical Report 11-025, University of Minnesota, Minneapolis, 2011.

[4] W. Groves and M. Gini, “An agent for optimizing airline ticket purchasing,” 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2013), St. Paul, MN, May 06 - 10, 2013, pp. 1341-1342.

[5] M. Papadakis, “Predicting Airfare Prices,” 2014.

[6] T. Janssen, “A linear quantile mixed regression model for prediction of airline ticket prices,” Bachelor Thesis, Radboud University, 2014.

[7] R. Ren, Y. Yang and S. Yuan, “Prediction of airline ticket price,” Technical Report, Stanford Univerisy, 2015.

[8] S. Haykin, Neural Networks – A Comprehensive Foundation. Prentice Hall, 2nd Edition, 1999.

[9] S.B. Kotsiantis, “Decision trees: a recent overview,” Artificial Intelligence Review, vol. 39, no. 4, pp. 261-283, 2013.

[10] G.B. Huang, Q.Y. Zhu and C.K. Siew, “Extreme learning machine: Theory and applications,” Neurocomputing, vol. 70, no. 1-3, pp. 489- 501, 2009.

[11] G.A. Papakostas, K.I. Diamantaras and T. Papadimitriou, “Parallel pattern classification utilizing GPU-Based kernelized slackmin algorithm,” Journal of Parallel and Distributed Computing, vol. 99, pp. 90-99, 2017.

[12] Aegean Airlines, https://en.aegeanair.com.

[13] https://github.com/humain-lab/airfare_prediction.

[14] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5-32, 2001.

[15] L. Breiman, J. Friedman, R. Olshen and C. Stone, Classification and Regression Trees. Boca Raton, FL: CRC Press, 1984.

[16] H. Drucker, C.J.C. Burges, L. Kaufman, A. Smola and V. Vapnik, "Support vector regression machines," Advances in neural information processing systems, vol. 9, pp. 155-161, 1997.